

MAE 3780 Robot Project Final Report Overview

Group Number: 54

Members: Faiza Huda (fh296), Apurva Hanwadikar (aah246), Zhou Xianyi (xz832)

1. Robot Design and Strategy Overview

Our original robot design focused on maximizing speed, using a semi-passive intake system, and retaining collected blocks effectively. The overall strategy was to outperform other robots with faster movement and efficient collection.

In our initial design phase, we aimed for a 3:1 gear reduction at the wheels to gear up the drivetrain. This would allow for higher driving speeds. A box made of thin sheet metal would be mounted to the front of the robot. The sheet metal would be thin enough that we could effectively slide it underneath the blocks. The collection system included a roller mechanism mounted to the top of our box. The roller is powered by the provided 360-degree microservo and spins continuously, guiding blocks into the robot while helping prevent collected blocks from falling out during movement.

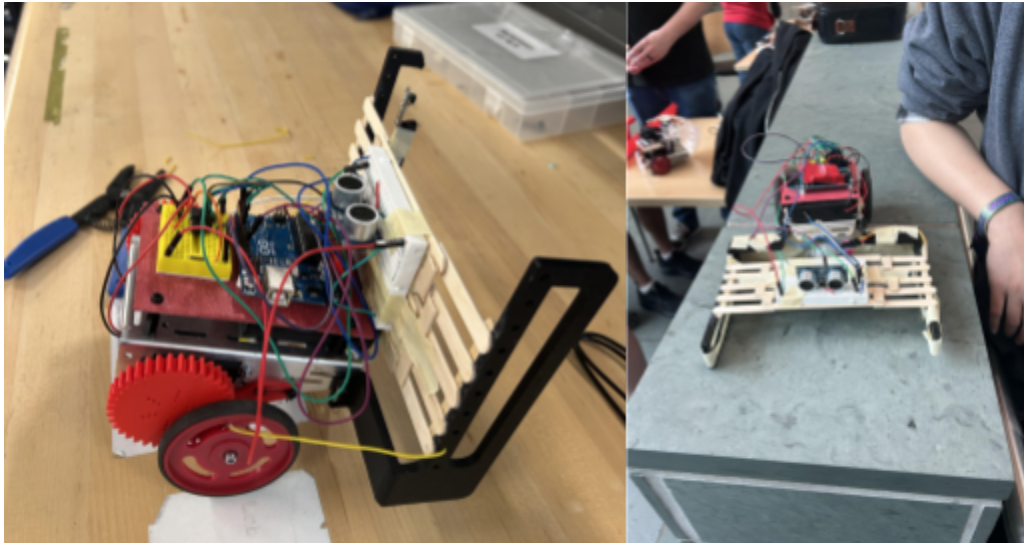
Electrically, the original design incorporated both a QTI sensor and an ultrasonic sensor. The QTI sensor would specifically detect the black border, while the ultrasonic sensor would assist in detecting the other robot and preventing collisions. The color sensor was not used since we wanted to maximize the number of cubes we could collect. We decided not to use the 9V battery because testing suggested that the 6V battery pack was enough power for the Arduino, servos, and sensors.

On the software side, the robot was planned using state modeling, as shown in the code (Appendix E). Different robot behaviors would operate as separate states, enabling an organized control logic that follows our strategy flowchart (Appendix D).

However, due to time and cost constraints, our mechanical design was simplified and made more robust. Our geartrain failed after repeated testing due to tolerancing issues and wear and tear. The final robot had straight mechanical arms that served as the retaining perimeter for the blocks. It was optimized for capacity while still fitting the bounding box, and would deploy when the servo wheels turned. We implemented the QTI but failed to use the ultrasonic sensor for anything other than decorative purposes.

Our strategy during the competition is to follow the left black border, reach the center of the arena where the blocks are concentrated, sweep across the board with our perimeter deployed, then, when detecting the border on the other end, turn 180 degrees

and run back along the center. Our strategy relied on our robot being faster than the opponent.



Picture of our robot, original idea (left), actual implementation (right)

2. Design Process Reflection.

The design for our robot was changed multiple times. Unfortunately, due to the complexity of the original idea, we went through iterations to simplify our final implementation.

We were limited by our budget, as we could not purchase many parts off the shelf. We attempted to make do with 3D-printed versions, but we quickly realized they were not strong or reliable enough, especially with the spur gears. They were prone to misalignment and skipping. After testing, we also realized that the 360 microservo would not spin fast enough to incorporate the roller intake idea, as it was slower than the robot's linear velocity. We decided to forgo them in the final design.

In milestone 3, we found it difficult to achieve a good QTI reading with the provided standoffs and the pre-existing chassis. It could not reach low enough to the ground for reliable feedback. While working through milestone 4, we realized we did not need a ceiling or a floor to our box, since our strategy was to always move forward while collecting the cubes. We prototyped with a cardboard border, but ultimately changed the collection mechanism into arms that the QTI sensors could mount to while being sufficiently low to the ground. A popsicle roof was added to support the extra circuitry for the ultrasonic sensor.

3. Competition Analysis.

Our robot did not perform particularly well during the competition due to several implementation pitfalls. Unfortunately, because the spur gears failed, we could not use our gear ratio to speed up our car. Hence, we were either on par with or slower than our opponents if they had purchased larger wheels. Our wheels were also not tested on the arenas, which led to some small deviations in accuracy for the distances that we travelled. That led to our software strategy not working very well, as we calibrated our motor timings with the faster speed in mind, as well as a different floor material. Moreover, because our sensors were not properly calibrated for the competition arena, our QTI sensors did not perform well for border detection. We could reliably follow the left border and move through the center, but often failed to stay within the arena. We lost some blocks that way, or our car would get stuck and be unable to turn back for a second run.

In terms of mechanical designs, our perimeter retained blocks fairly well. The arms provided a good boundary for collecting and guiding the blocks across the board. For the most part, though, every match was unpredictable, as many external factors cannot be accounted for. We often experienced our robots and the opponents' pushing each other off track or getting stuck within each other's mechanisms. Winning was also dependent on how well the other team could retain its blocks. As a result, you could be winning one moment and find yourself losing the next.

4. Conclusions.

In conclusion, our design, while thoughtful, was too complex to be fully effective during the competition. We could have begun with a much simpler design with cheaper materials. The competition was a lot less aggressive than we expected, and we did not need as robust a perimeter for the blocks as we expected. For simpler and quicker implementation, We would have not used the gear ratio for higher speed and used bigger wheels instead. We should also add a back wall to our perimeter on the car because a lot of the cubes were lost that way. Many of our ideas were also heavily limited by time. Hence, we would advise future students doing the project next year to start early and go through the trial-and-error process rigorously.

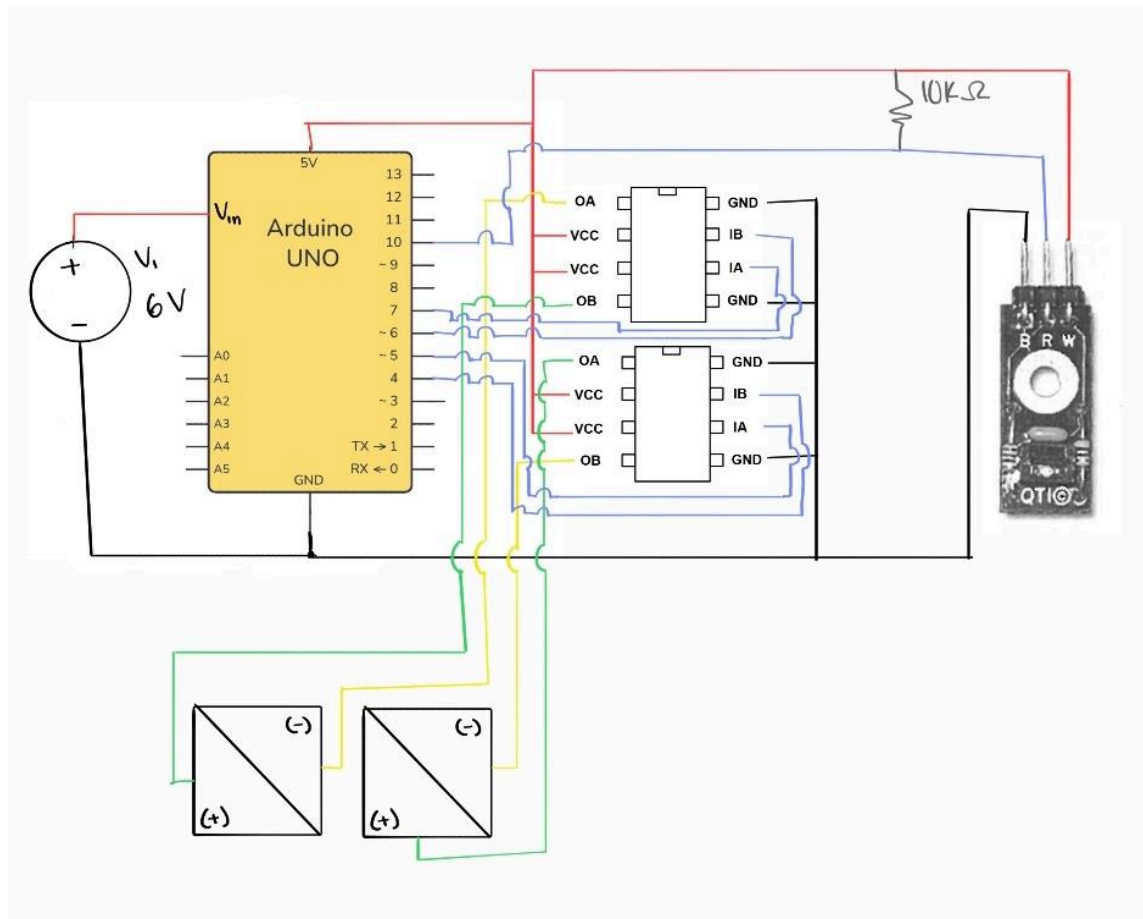
5. Appendix A: bill of materials (BOM).

We used the provided screws, standoffs and electrical tape from the lab. For our final design, we used the collecting arms and the popsicle sticks. Small gears, big gears and bearings were used for prototyping. The popsicle sticks and bearings were found as scrap parts but evaluated as newly purchased from their sources.

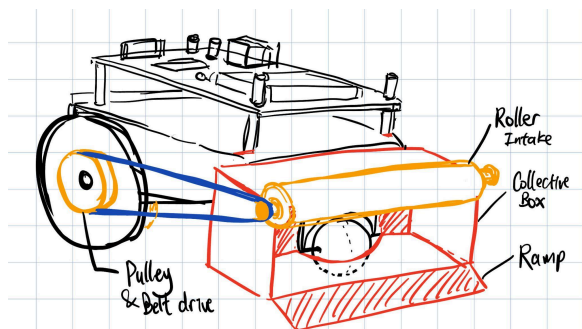
<u>Item</u>	<u>Quantity</u>	<u>Source</u>	<u>Unit Mass</u>	<u>Unit Price</u>	<u>Total Price</u>
Collecting Arms	2	3D Print	14.07g	6.628	13.256
Popsicle Sticks	5	Scavenged (Walmart Hello Hobby Wood Craft Sticks, 50-Pack)	N/A	0.02 for 1	1.24 for 50
Small gears	2	3D Print	1.71g	1.684	3.368
Big gears	2	3D Print	8.64g	4.456	8.912
Bearings	2	Scavenged (Digikey 1188-BEARING-M F84ZZ-ND)	N/A	1.08	4.32 for 4

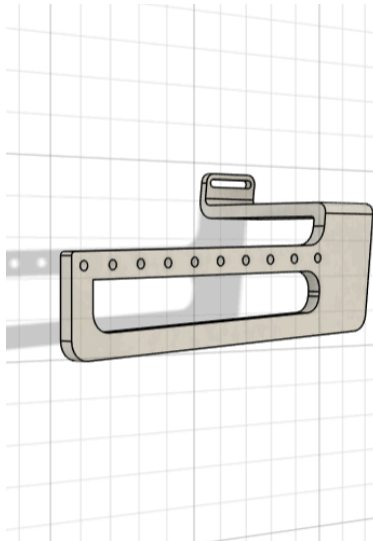
Total Price: \$ 31.096

6. Appendix B: circuit diagram. The circuit diagram is as follows: it uses two L1910 H-Bridges to control the motors and a QTI sensor to detect the black border.

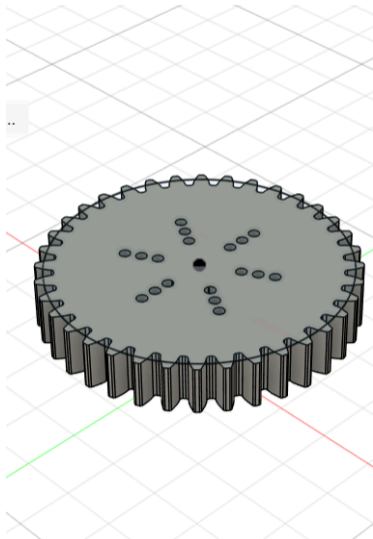


7. Appendix C: CAD files and drawings.

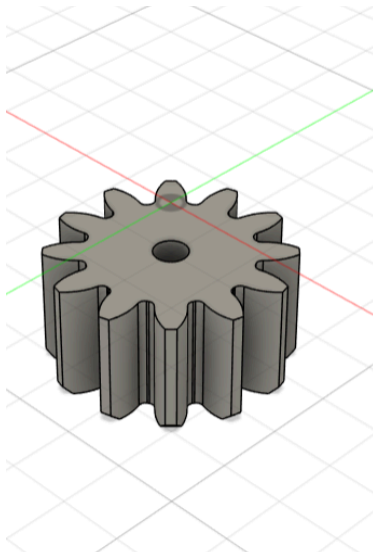




PROPERTIES	
Bodies (1)	
Area	13887.786 mm ²
Density	0.001 g / mm ³
Mass	21.864 g
Volume	23510.098 mm ³
Physical Material	Paper
Appearance	Paper (White)
Bounding Box	
Center of Mass	68.161 mm, -124.099 m...
Moment of Inertia at Center of Mass (g l...	



PROPERTIES	
Bodies (1)	
Area	8437.836 mm ²
Density	0.001 g / mm ³
Mass	18.542 g
Volume	19937.521 mm ³
Physical Material	Paper
Appearance	Opaque(160,160,160)
Bounding Box	
Center of Mass	-1.659E-05 mm, 3.077E...
Moment of Inertia at Center of Mass (g l...	
OK Cancel	



PROPERTIES	
Bodies (1)	
Area	1690.338 mm ²
Density	0.001 g / mm ³
Mass	2.31 g
Volume	2484.094 mm ³
Physical Material	Paper
Appearance	Opaque(160,160,160)
Bounding Box	
Center of Mass	0.00 mm, 0.00 mm, -9...
Moment of Inertia at Center of Mass (g l...	

The volumes are screenshots taken off of Fusion 360.

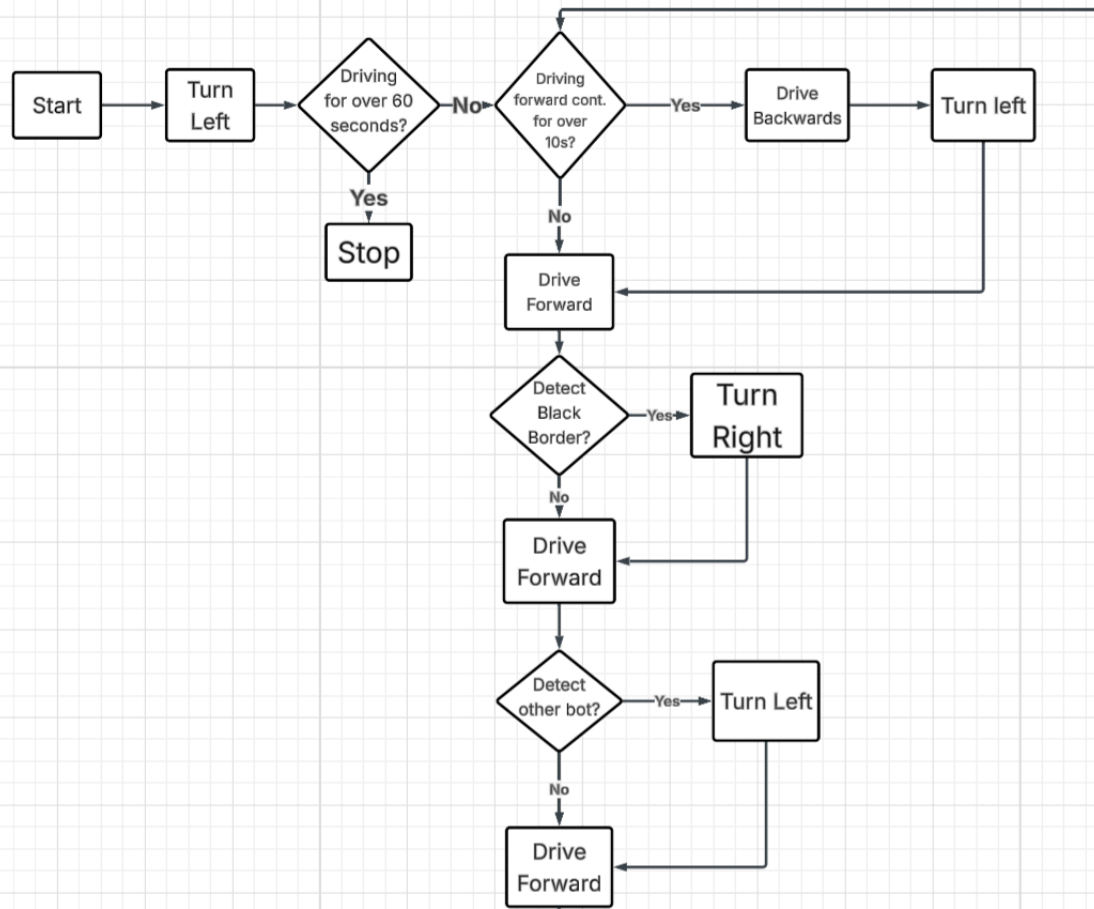
Weight calculation based on 75% PLA density:

- Small gear: $0.75 \times 1.25 \times (2484.094 / 10^3) = 2.32\text{g}$
- Big gear: $0.75 \times 1.25 \times (19937.521 / 10^3) = 18.69\text{g}$
- Arms: $0.75 \times 1.25 \times (23510.098 / 10^3) = 22.04\text{g}$

Actual masses:

- Small gear: 1.71g
- Big gear: 8.64g
- Arms: 14.07g

8. Appendix D: Flowchart. This is the intended algorithm.



9. Appendix E: Code commented Arduino code. A clear understanding of each line of code should be demonstrated.

Sensor code with QTI and ultrasonic implementation (ultrasonic ultimately not used):

```
#include <Arduino.h>
```

```
// --- Motor Functions ---
```

```
void stop(){  
  PORTD = 0b00000000; //sets all motor pins to 0, causes the motors to stop spinning  
}
```

```
void drive_forward(){  
  PORTD = 0b01100000; //sets pins 6 and 5 to high, outputs voltage in the forward  
  //direction (positive) to spin both motors forward  
}
```

```
void drive_backward(){  
  PORTD = 0b10010000; //sets pins 7 and 4 to high, outputs voltage in the backward  
  //direction (negative) to spin both motors backward  
}
```

```
void turn_left(){  
  PORTD = 0b00110000; //spins the right motor forward, left motor backward and sets  
  //those corresponding pins to high  
}
```

```
void turn_right(int time){  
  PORTD = 0b11000000; //spins the left motor forward, right motor back and sets  
  //those corresponding pins to high  
}
```

```
void forward(int time){  
  PORTD = 0b01100000; //spins both motors forward for a fixed time  
  _delay_ms(time);  
  stop();  
  _delay_ms(10);  
}
```



```

void backward(int time){
    PORTD = 0b10010000; //spins both motors backward for a fixed time
    _delay_ms(time);
    stop();
    _delay_ms(10);
}

```

```

void left(int time){
    PORTD = 0b00110000; //turn left for a fixed time
    _delay_ms(time);
    stop();
    _delay_ms(10);
}

```

```

void right(int time){
    PORTD = 0b00110000; //turn right for a fixed time
    _delay_ms(time);
    stop();
    _delay_ms(10);
}

```

// --- FSM States — using a finite state machine to track the states the robot is in to determine what the next course of action should be

```

enum State {
    FORWARD,
    TURN_LEFT_BLACK,
    BACKWARD_STUCK,
    TURN_LEFT_DELAYED,
    STOPPED
};

```

State state = FORWARD; //initially the robot starts moving forward

```

// --- Timing variables ---
unsigned long start_time;
unsigned long state_start;
unsigned long forward_start;

```

```

unsigned long forward_since_black_start;

volatile int period = 0;
volatile bool newReading = false;

bool sonar_activated;

bool first_black = false;
bool delayed_turn_done = false;

void setup(){
    init();
    Serial.begin(9600);
    start_time = millis();
    DDRD = 0b11110000; //set pins 4-7 (motor pins) to output
    sonar_activated = false; //sonar used for detecting other robots, not initially necessary

    //timing variables
    start_time = millis();
    state_start = millis();
    forward_start = millis();
}

void sonor_init() //initializes the sonar sensor on pin 12
{
    DDRB &= ~0b00010000;
    PORTB &= ~0b00010000;
    TCCR1A = 0; //normal mode for timer 1
    TCCR1B = (1 << CS11);    // set the prescaler to 8
}

void loop(){

    unsigned long now = millis();
    // the game ends after a minute, so after that time has passed, we stop the state
    machine
    if (now - start_time >= 60000){
        state = STOPPED;
    }
}

```

```

switch(state){

    case FORWARD:
        forward(100); //go forward for 100ms
        int dist = 1000; //initialize to an arbitrarily large number so we don't hit this condition
//early
        if (sonar_activated) {
            dist = get_distance_cm(); //get the distance away from the object if we're using
//sonar
        }
        if (dist > 3) {
            //we carry on as if sonar doesn't exist - object is too far, or we haven't started sonar
            // black detected
            if (PIND & 0b00000100){ //if QTI sensor detects a black border, we go into turn
//state
                Serial.println("BLACK");
                state = TURN_LEFT_BLACK;
                state_start = now;
            }

            // stuck (15s forward) - if we've been going forward for a while with nothing
//detected, we're doing something wrong, so reverse
            else if (now - forward_start >= 15000){
                Serial.println("STUCK");
                state = BACKWARD_STUCK;
                state_start = now;
            }

            // delayed turn (10s after first black)
            else if (first_black && !delayed_turn_done &&
                (now - forward_since_black_start >= 10000)){
                Serial.println("DELAYED TURN");
                state = TURN_LEFT_DELAYED;
                state_start = now;
            }
        } else {
            //we've detected something in the distance threshold, and we want to turn away
            turn_right(470);
            state = FORWARD;
        }
}

```

```
break;
```

```
    //if a black line has been detected, we turn left  
case TURN_LEFT_BLACK:  
    Serial.println("GO");  
    left(470);
```

```
    if (!first_black){  
        first_black = true;  
        forward_since_black_start = now;  
    }  
}
```

```
if (now - state_start >= 470){  
    stop();  
    forward_start = now;  
    state = FORWARD;  
}  
break;
```

```
    //if stuck, we go backward for a bit, and if we've been in the backward state for  
//a bit we go into the turning state to get out of the way
```

```
case BACKWARD_STUCK:  
    backward(20);  
  
    if (now - state_start >= 500){  
        state = TURN_LEFT_BLACK;  
        state_start = now;  
    }  
    break;
```

```
case TURN_LEFT_DELAYED:  
    left(470);
```

```
    if (now - state_start >= 500){  
        stop();  
        delayed_turn_done = true;  
        forward_start = now;  
        state = FORWARD;  
        sonor_init();
```

```

        sonar_activated = true;
    }
    break;

case STOPPED:
    stop();
    Serial.println("STOPPED");
    while(1); // halt, loop is caught here
    break;
}
}

void send_trigger()
{
    cli();                // disable interrupts during trigger
    DDRB |= 0b00010000;   // switch to output
    PORTB &= ~0b00010000;
    _delay_us(2);
    PORTB |= 0b00010000;   // pulse high
    _delay_us(5);
    PORTB &= ~0b00010000;   // pull low
    DDRB &= ~0b00010000;   // switch back to input
    PORTB &= ~0b00010000;   // no pull-up

    PCIFR |= 0b00000001;   // clear any pending PCINT flag before enabling
    PCICR |= 0b00000001;
    PCMSK0 |= 0b00010000;
    sei();                // re-enable interrupts
}

int get_distance_cm()
{
    newReading = false;
    send_trigger();

    unsigned long timeout = millis();
    while (!newReading) {
        if (millis() - timeout > 50) {
            break;
        }
    }
}

```

```

    }
}

// disarm interrupt until next trigger
PCMSK0 &= ~0b00010000;

return period / 116;
}

ISR(PCINT0_vect)
{
    if (PINB & 0b00010000) {
        TCNT1 = 0;
    } else {
        period = TCNT1;
        newReading = true;
    }
}

```

Hardcoded version (same as sensor version except with loop substituted as following):

```

void loop(){
    // //Hard Coded Version (must check distances)
    drive_forward(3456);
    turn_left(550);
    drive_forward(3949);
    turn_left(550);
    drive_forward(6912);
    turn_left(550);
    drive_forward(3949);
    turn_left(550);
    drive_forward(3456);
    while (1){
        }
    }
}

```